

JOHANSEN CO₂ STORAGE

Bayesian Optimization of Individual Injector Wells

Technical Documentation

johansen_bo_individual_wells_v5_boundary_scan.ipynb

Toolchain: Python 3 · Jupyter · Optuna TPE ↔ MATLAB · MRST 2026a · CO2lab

Optimization: 120 trials · 36 warm-up · 15-D search space · seed=42 · SQLite-persisted

Best result: 196.8 Mt CO₂ stored · 317.94 bar peak BHP · 0 boundary cells breached (Trial #56)

1. Project Overview

1.1 The Problem

This project designs an injection schedule for a CO₂ storage site in the **Johansen formation** — a saline aquifer in the northern North Sea and one of the standard benchmark reservoirs distributed with **MRST** (the MATLAB Reservoir Simulation Toolbox) and its **CO2lab** module for carbon capture and storage (CCS) research.

Five candidate injector wells are available. For each well, the operator must choose:

- **Q** — injection rate (Mt CO₂/yr)
- **T_start** — year injection begins
- **T_end** — year injection stops

That gives 15 decision variables in total (5 wells × 3 parameters). The goal is to store as much CO₂ as possible without violating two physical safety limits:

- **Caprock fracture risk:** injecting too fast raises bottom-hole pressure (BHP) until it risks fracturing the seal above the reservoir.
- **Containment risk:** injecting too much/too early pushes the CO₂ plume to the edge of the modelled footprint before it can dissolve or trap.

Each candidate configuration can only be evaluated by running a **full MRST reservoir simulation** — a 1000-year multiphase flow simulation over a 100×100×11 grid (29,128 active cells) — which is comparatively expensive and must be triggered **manually in MATLAB**. This combination of expensive black-box evaluation, no gradient information, a human required in the loop, mixed continuous/integer parameters, and nonlinear physical constraints is exactly the setting **Bayesian Optimization (BO)** was designed for.

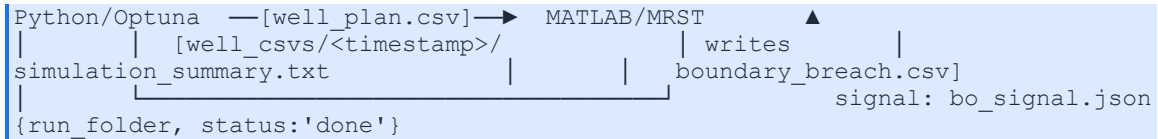
1.2 CCS Context

Carbon Capture and Storage (CCS) is the practice of capturing CO₂ from industrial or atmospheric sources and injecting it into deep geological formations for permanent storage. Saline aquifers like Johansen are attractive targets because they are large, widespread, and not in competition with hydrocarbon production.

The central engineering risk is injectivity vs. containment: inject fast enough to be economical, but not so fast that the reservoir over-pressurizes (risking caprock fracture) or the CO₂ plume migrates beyond the area that has been characterized and is being monitored.

2. Architecture & Design

The notebook does **not** run the reservoir simulator itself — MRST/CO2lab only runs inside MATLAB. Instead, the notebook implements a **file-based, human-in-the-loop handshake** between two processes.



Why this design, not a fully-automated loop? MRST/CO2lab only runs inside a MATLAB session, and the notebook has no way to invoke MATLAB programmatically in this setup. Cell 7 implements a handshake: it writes the proposed configuration, prints an instruction to run the simulation, then polls for evidence that a new result exists — either a `bo_signal.json` file (written by the MATLAB script when it finishes) or a new complete folder under `well_csvs/`. Whichever arrives first is accepted.

2.1 Key Files

File	Written by	Purpose
<code>well_plan.csv</code>	Python	Live well configuration MATLAB reads for the next simulation. Fully regenerated each trial from the backup template + 15 trial parameters.
<code>well_plan_BACKUP_BO.csv</code>	Python (once)	Frozen snapshot of the initial <code>well_plan.csv</code> . Immutable template for all non-optimized wells and metadata.
<code>bo_pending.json</code>	Python	Parameters just written + timestamp — lightweight 'trial in flight' marker, deleted once the trial resolves.
<code>bo_signal.json</code>	MATLAB	<code>{run_folder, timestamp, status:'done'}</code> — tells Python which output folder to read; deleted by Python on consumption.
<code>well_csvs/DD_MM_YYYY__HH_MM/</code>	MATLAB	One folder per simulation run containing <code>simulation_summary.txt</code> , <code>boundary_breach.csv</code> , per-well CSVs, and diagnostic PNGs.
<code>bo_v6_results.json</code>	Python	Full trial log (every parsed trial, in order). Rebuilt from scratch on every notebook run.
<code>johansen_bo_v6_4wells.db (SQLite)</code>	Optuna	Persisted optuna.Study — also rebuilt from scratch each run.

2.2 write_well_plan() and the Backup Template

A critical detail: `write_well_plan()` always reads its starting point from `well_plan_BACKUP_BO.csv`, not from the current `well_plan.csv`.

```

def write_well_plan(well_params):
    df = pd.read_csv(BACKUP_CSV)
    always start from the frozen template    for well_name, wp in
    well_params.items():                    ...
    only the 5 optimised wells              df.to_csv(WELL_PLAN_PATH, index=False)

```

So `well_plan.csv` is fully regenerated on every trial from a fixed base plus the 15 parameters for that trial. This prevents any drift or compounding edits across 120 trials — each `well_plan.csv` is a deterministic function of `(BACKUP_CSV, trial params)`.

2.3 Source of Truth: the Disk, Not the Cache

SOURCE OF TRUTH: `well_csvs/` folders on disk — always. `bo_v4_results.json` and `SQLite` are **DERIVED** caches, rebuilt from scratch every run.

Every time the notebook is restarted, Cell 6 re-scans **all** folders under `well_csvs/`, re-parses every `simulation_summary.txt` / `boundary_breach.csv`, rebuilds `trial_log` from scratch, rewrites `bo_v6_results.json`, deletes the old SQLite file, and replays every historical trial back into a brand-new `optuna.Study` via `study.add_trial(...)`. This makes the notebook robust to kernel restarts, crashes, or edits to the reward function — nothing is trusted except the raw simulation outputs on disk.

3. Cell-by-Cell Breakdown

The notebook has 12 cells (1 markdown + 11 code). Version labels are inconsistent within the file itself — the filename says v5, the in-notebook banner says v4, and Cell 2's own comments say v6 (see §6 for the full explanation).

Cell	Type	Role
0	Markdown	Project banner: wells, 3 conditions, workflow diagram, changes from v3
1	Code	Install dependencies (optuna, scikit-learn, joblib)
2	Code	Configuration: paths, physical limits, search-space bounds — the single most important cell
3	Code	Utility functions: I/O helpers, parsers, reward function
4	Code	Live 3-panel progress chart (render_live_chart())
5	Code	Backup + pre-flight verification
6	Code	Smart startup — rebuild trial_log + Optuna study from disk
7	Code	The optimization loop itself (handshake mode)
8	Code	Best-trial report; writes winner back to well_plan.csv
9	Code	Full trial summary table
10	Code	Optuna parameter-importance analysis (fANOVA-based)
11	Code	Gradient-Boosting surrogate model export (CO ₂ & BHP, via joblib)

Cell 0 — Markdown Banner

Documents the 5 wells, the 3 physical conditions and their sources/limits, an ASCII workflow diagram, and a 'changes from v3' list. This is the closest thing the notebook has to a changelog, though it describes the v4 state, not the current v6 configuration in Cell 2.

Cell 1 — Install Dependencies

Installs optuna, scikit-learn, and joblib if missing (idempotent — checks `__import__` first). No project-specific logic.

Cell 2 — Configuration

The single most important cell — everything else reads from constants defined here: all file paths rooted at `MRST_ROOT`, physical constraints (`BHP_LIMIT_BAR = 334.8` bar, `S_CO2_THRESHOLD = 0.15`, `MAX_INJECTION_YEARS = 110`), `BHP_PENALTY_WEIGHT = 3.0`, `Q_MAX_MT_YR = 2.0`, the 5-well `WELL_NAMES` roster, and the 15-D `BOUNDS` dict.

Cell 3 — Utility Functions

Defines `backup_well_plan()` / `restore_well_plan()`, `write_well_plan()`, `get_latest_run_folder()`, `hash_params()`, `parse_per_injector_bhp()`, `parse_boundary_breach()`, `parse_co2_total()`, `compute_reward_and_violations()`, and small I/O helpers.

Cell 4 — Live Progress Chart

Defines `render_live_chart()` — a 3-panel matplotlib dashboard redrawn after every trial. Panel 1: CO₂ per trial (colour-coded by feasibility). Panel 2: per-trial max BHP vs. the 334.8 bar limit. Panel 3: boundary-breach percentage per trial.

Cell 5 — Backup & Verify Setup

Runs `backup_well_plan()`, prints the active wells from the backup CSV, checks all 5 optimised wells are present, and — if a prior run folder exists — parses and prints its results as a smoke test that the parsers still match the current `simulation_summary.txt` format. Also clears any stale `bo_pending.json`.

Cell 6 — Smart Startup

Implements the 'source of truth' principle: scans every `well_csvs/*` folder, parses each into a full trial-log entry (re-deriving `is_feasible/reward` via the current reward function — so reward-function edits are automatically back-applied), writes `bo_v6_results.json`, deletes and rebuilds the SQLite study, and replays every trial via `study.add_trial()`. Ends by printing how many trials remain.

Cell 7 — Optimization Loop (Handshake Mode)

The core loop. For each remaining trial: (1) `study.ask()` proposes a new point; (2) `Q` is sampled with `step=0.05`, then `T_start`, then `T_end` with its lower bound pinned to `T_start + MIN_INJECTION_DURATION` (the v4.1 fix eliminating wasted trials); (3) `write_well_plan()`, `write_pending()`, `clear_signal()` prepare the proposal; (4) Cell 7 polls every 5 s (30-min timeout) for either `bo_signal.json` or a new completed `well_csvs/` folder; (5) on timeout → `study.tell(FAIL)`; (6) on success → `parse`, `compute_reward`, `study.tell(reward)`, refresh chart. At loop end, prints the best feasible trial.

Cell 8 — Best Trial Report

Finds the best feasible trial by reward (= CO₂ stored), prints its per-well parameters in a table, and writes that configuration back into `well_plan.csv` ready for a final confirmatory MATLAB run.

Cell 9 — Trial Summary Table

Aggregate stats across all OK trials: breach counts/rates per condition, CO₂/BHP ranges, per-well `Q` ranges explored. Renders a styled pandas table of the top-20 trials (feasible = green, infeasible = red).

Cell 10 — Parameter Importance

Calls `optuna.importance.get_param_importances()` (fANOVA-based) and plots a horizontal bar chart colour-coded by well, saved as `bo_v4_importance.png`. Requires at least `N_WARMUP + 1` completed trials.

Cell 11 — Surrogate Model Export

With ≥ 10 completed trials, fits two `GradientBoostingRegressor` models (200 trees, depth 4) — one predicting `co2_mt`, one predicting `max_bhp_bar` — from the 15 raw parameters after `StandardScaler` normalization. Exports both models plus the scaler via `joblib`. Gives a cheap, instant approximation of the expensive MRST simulation for offline what-if exploration.

4. Optimization Problem Formulation

4.1 Decision Variables

15 mixed continuous/integer variables — 3 parameters per well across 5 independent wells:

Variable	Type	Bounds	Notes
Q_w (Mt/yr)	Continuous, step 0.05	[0.0, 2.0]	Allows Q = 0 (well disabled). Ceiling cut from 5.0 to 2.0 after 0/120 feasible trials — see §6
T_start_w (yr)	Integer	[0, 105]	Year injection begins
T_end_w (yr)	Integer	[T_start_w + 5, 110]	Dependent lower bound pins a valid ≥ 5 -yr window by construction (v4.1 fix)

4.2 Objective

Maximise: CO₂ injected (Mt) — reported by MATLAB as 'Total CO2 injected' in simulation_summary.txt

4.3 Constraints

#	Condition	Limit	Data Source
1	Caprock seal — per-injector BHP	≤ 334.8 bar (7% safety margin below 360 bar fracture pressure)	BHP_WELL lines in simulation_summary.txt
2	Boundary CO ₂ containment	S_CO ₂ < 15% on every scanned western-boundary cell	boundary_breach.csv — MATLAB applies threshold, Python reads verdict
3	Injection window	T_end ≤ 110 yr	Enforced structurally by search-space bounds — cannot be violated by construction

⚠ BHP limit discrepancy in source code comments: Cell 2 and Cell 0 state ' ≤ 324 bar (10% margin)' — this is stale documentation from an earlier BHP_SAFETY_MARGIN = 0.10 configuration. The code as written computes $\text{BHP_LIMIT_BAR} = 360 \times (1 - 0.07) = 334.8$ bar. Verified by back-computing from bo_v6_results.json: every entry satisfies $\text{max_bhp_bar} - \text{bhp_violation} = 334.80$ exactly. Treat any '324 bar' reference in the comments as stale — the real, enforced limit is 334.8 bar (≈ 335 bar).

4.4 Penalization Scheme

```
bhp_violation      = max(0, max_bhp_bar - 334.8) boundary_violation =
breach_fraction    = # 0-1, fraction of boundary cells breached
is_feasible        = (bhp_violation == 0) and (not breach_detected) if
is_feasible:       reward = co2_mt # maximise
CO2 stored else:   reward = -(3.0 × bhp_violation / 334.8 +
boundary_violation) × 1000
```

This is a standard constrained-BO pattern: every feasible trial outranks every infeasible trial, because feasible rewards are small positive numbers (41–365 Mt observed in uploaded results) while infeasible rewards are large negative numbers (as low as roughly -2,600 in a 916-bar worst case).

Among infeasible trials, the reward still provides a useful gradient — a smaller violation scores less negatively than a larger one — so the surrogate can still learn to move toward the feasible boundary rather than treating all infeasible trials as equally bad.

The **3.0× weight** on the BHP term reflects the project's judgment that caprock-fracture risk is more hazardous than a boundary-saturation exceedance. BHP_PENALTY_WEIGHT was raised from 1.0 to 3.0 at trial 36 of an earlier 150-trial run, after a visual analysis showed certain wells breaching BHP by 50–60% with no pressure-relief path. The `constraints_func` in Optuna's TPE sampler separately mirrors `bhp_violation` and `boundary_violation` as a constraint vector, biasing proposals toward the feasible region in addition to the reward shaping — the two mechanisms reinforce each other.

5. Data Flow & File I/O

5.1 Inputs

- `well_plan.csv` — full 40-row well roster; only 5 rows are touched per trial.
- `well_plan_BACKUP_BO.csv` — the frozen template (§2.1); created once and never overwritten by the notebook thereafter.

5.2 Outputs

File	Written by	Contents
<code>bo_v6_results.json</code>	Python	Every parsed trial (params + all reward/violation fields), rebuilt from disk each run
<code>bo_signal.json</code>	MATLAB	{ <code>run_folder</code> , <code>timestamp</code> , <code>status</code> } — polling handshake
<code>bo_pending.json</code>	Python	Currently in-flight trial's raw parameters (deleted on resolution)
<code>johansen_bo_v6_4wells.db</code>	Optuna/SQLite	Full study state, rebuilt from disk each run
<code>boundary_breach.csv</code> (per run folder)	MATLAB	<code>cell_x</code> , <code>cell_y</code> , <code>breach</code> (0/1) for every scanned boundary cell
<code>simulation_summary.txt</code> (per run folder)	MATLAB	Human-readable run report: config, fluid model, active wells, mass balance, per-injector BHP, boundary scan summary
<code><well>.csv</code> (per run folder)	MATLAB	Time series of per-layer pressure/saturation/depth at each well location
saturation / pressure / performance PNGs	MATLAB	Diagnostic visualisations at injection-end and simulation-end
<code>johansen_v4_co2_surrogate.pkl</code> , <code>johansen_v4_bhp_surrogate.pkl</code> , <code>johansen_v4_scaler.pkl</code>	Python (Cell 11)	Exported surrogate models for offline use

5.3 Folder Naming Convention

Each simulation run gets its own folder under `well_csvs/`, named **DD_MM_YYYY__HH_MM** — e.g. `04_08_2026__20_21` for 20:21 on 4 August 2026. This format sorts correctly as a plain string, which is exactly what `get_latest_run_folder()` relies on (`sorted(glob(...))[-1]`), and matches the glob pattern `??_??_????_??_??` used throughout the notebook.

6. Key Design Decisions & Version History

v3 → v4 — Clustering removed

Every well independently optimised. Moved from a 6-D to a 15-D search space (5 wells × 3 parameters). Added 31/07/01 to the roster, replaced 31/05/02. Widened Q to a free range of (0.0, 5.0 Mt/yr). Opened the full time window ($T_{\text{start}} \in [0, 95]$, $T_{\text{end}} \in [5, 100]$). Run for 150 trials with 45 warm-up.

v4 → v4.1 — Timing-sampling bug fix

Independent sampling of T_{start} and T_{end} allowed $T_{\text{start}} \geq T_{\text{end}}$ on a large share of trials, silently wasting trial budget slots before MATLAB was ever run. Fixed by sampling T_{end} after T_{start} with a dependent lower bound (T_{end} lower bound pinned to $T_{\text{start}} + \text{MIN_INJECTION_DURATION}$). A safety-net check (`any_bad_timing`) remains afterward but should be mathematically unreachable.

v4 → v5 — Boundary scan upgraded

Replaced 4 hand-picked `SBoundary_test_well` point samples with a full-boundary scan in MATLAB: every westernmost active cell per J-row at the topmost perforated layer is checked against the 15% S_{CO_2} threshold and written out as `boundary_breach.csv`. The threshold check itself moved from Python into MATLAB — Python now only reads MATLAB's verdict. This is the version referenced in the notebook's filename.

v5 → v6 — Well 31/2-5 retired; Q ceiling tightened

After a 120-trial v4 run came back with 0/120 feasible trials, two changes were made together: (1) 31/2-5 was dropped from the optimised roster — it alone accounted for the worst BHP breaches (up to 525 bar, attributed to geographic isolation with no pressure-relief path, and no fix was found); (2) the per-well Q ceiling was cut from 5.0 to 2.0 Mt/yr, which finally opened up a feasible region. The uploaded `bo_v6_results.json` shows 34/120 feasible trials under this configuration, with a best feasible result of 196.8 Mt CO₂.

⚠ Version-label drift: The file is named `_v5_boundary_scan.ipynb`; the Cell 0 banner says 'v4'; Cell 2's header says 'v6 — 4 Wells, $Q \leq 2.0$ Mt/yr'; and Cell 2 prints '12 params × 10 per param (4 wells × 3)'. None of these agree. `WELL_NAMES` actually contains 5 wells, `BOUNDS` produces 15 parameters (matching $N_{\text{TRIALS}} = 120 = 15 \times 8$ and every entry in `bo_v6_results.json`). Trust the code (`WELL_NAMES`, `BOUNDS`, the results file) over the printed strings and comments when they disagree. Worth a cleanup pass before handing off or reusing this notebook.

7. Optuna / TPE Sampler Setup

```
sampler = TPESampler(n_startup_trials=36, seed=42,
constraints_func=optuna_constraints_func) study = optuna.create_study(
study_name='johansen_co2_bo_v4_individual', direction='maximize',
sampler=sampler, storage='sqlite:///.../johansen_bo_v6_4wells.db',
load_if_exists=False, # always rebuilt from disk )
```

- **n_startup_trials = 36** (30% of the 120-trial budget). Trials 0–35 are sampled quasi-randomly across the 15-D space before the TPE surrogate takes over. Deliberately generous — 15 dimensions need a dense initial sample before a tree-structured density-ratio model can say anything useful.
- **seed = 42** — reproducible warm-up sampling.
- **constraints_func** — returns [bhp_violation, boundary_violation] per trial. Optuna's TPE sampler uses this to bias proposals toward the feasible region, independently of and in addition to the reward-shaping penalty.
- **load_if_exists = False** combined with `os.remove(_db_path)` beforehand — the study is never resumed from the SQLite file directly; it is always rebuilt fresh from `well_csvs/` each run, then persisted so that a crash mid-session does not lose progress within that session.
- **Distributions for replay** — Cell 6 reconstructs `FloatDistribution(*bounds, step=0.05)` for every `Q_*` and `IntDistribution(*bounds)` for every `T_start_*/T_end_*` before replaying historical trials in via `study.add_trial()`.

8. Live Progress Dashboard

`render_live_chart()` (Cell 4) redraws a 3-panel matplotlib figure after every completed trial, and saves it to `bo_v4_live_progress.png`:

Panel 1 — CO₂ per trial

Scatter of CO₂ stored per trial, colour-coded into four categories: ● feasible, ● BHP breach only, ● CO₂ leak only, ● both. A dashed line tracks the running best CO₂ value across all trials (feasible or not), giving visibility into where the optimizer is exploring.

Panel 2 — Condition 1 (BHP)

Bar chart of each trial's max injector BHP. Bars coloured red above the 334.8-bar limit and blue below, with horizontal reference lines at 334.8 bar (operational limit) and 360 bar (fracture pressure).

Panel 3 — Condition 2 (boundary breach)

Scatter of the boundary-breach percentage per trial (green dot at 0%, red dot if any cell breached), annotated with 'breached/total' cell counts wherever a breach occurred.

9. How to Run — One Full BO Iteration

Step 1 Cells 1–4 (once per kernel session)

Install dependencies, load configuration, define utility functions, and define the chart renderer. These cells are idempotent — safe to re-run.

Step 2 Cell 5 — Backup & verify

Backs up `well_plan.csv` (first time only) and verifies the parsers still understand the most recent `simulation_summary.txt` on disk, if any. Clears any stale `bo_pending.json`.

Step 3 Cell 6 — Smart startup

Re-scans every `well_csvs/*` folder on disk, rebuilds `trial_log` and the Optuna study from scratch, and reports how many of the 120 trials remain.

Step 4 Cell 7 — Interactive loop

For each remaining trial: (a) Cell 7 prints the proposed `Q/T_start/T_end` for all 5 wells and writes them into `well_plan.csv`. (b) Switch to MATLAB and run the Johansen CO₂ simulation script — it reads `well_plan.csv`, runs the 1000-year MRST simulation, and writes a new timestamped folder under `well_csvs/`. (c) Return to the notebook. Cell 7 detects the new folder (via `bo_signal.json` or direct polling), parses the results, computes the reward, records the trial, refreshes the live chart, then automatically proposes the next trial. No re-execution needed between trials; it loops internally until `N_TRIALS = 120` or a keyboard interrupt.

Step 5 Cell 8 — Best trial

Once satisfied (or after all 120 trials complete), report the best feasible trial and write its configuration back into `well_plan.csv` for a final confirmatory MATLAB run.

Step 6 Cells 9–11 (optional)

Full trial table, parameter-importance plot, and fast surrogate model export for offline what-if analysis. Can be run at any time after enough trials exist.

10. Glossary

Term	Definition
BHP	Bottom-Hole Pressure — the fluid pressure at the base of a well; the key operational constraint against caprock fracture.
S _{CO₂}	CO ₂ (gas-phase) saturation — the fraction (0–1) of pore volume in a grid cell occupied by CO ₂ rather than brine.
Mt/yr	Megatonnes (million metric tons) per year — standard CCS injection-rate unit.
Caprock	The low-permeability rock layer sealing the top of the storage reservoir, preventing CO ₂ from migrating upward; its mechanical integrity is what the BHP constraint protects.
Fracture pressure	The pressure above which the caprock risks mechanical failure (360 bar in this model).
Plume migration	The lateral/vertical spread of the buoyant CO ₂ plume through the reservoir after injection stops, tracked here over a 1000-year post-injection horizon.
CCS	Carbon Capture and Storage.
TPE	Tree-structured Parzen Estimator — Optuna's default Bayesian-optimization strategy; models the ratio of 'good' vs. 'bad' parameter densities rather than fitting a direct Gaussian-process regression of the objective.
Optuna	The open-source hyperparameter/black-box optimization framework driving the search loop, with SQLite-backed persistence used here.
Boundary breach	When S _{CO₂} exceeds the 15% threshold at the modelled reservoir's outer edge — a proxy for the plume escaping the characterized/monitored area.
MRST	MATLAB Reservoir Simulation Toolbox (SINTEF) — the open-source subsurface flow simulator this project depends on.
CO2lab	MRST's dedicated CCS module, providing standard formations (including Johansen) and CO ₂ -brine flow physics.
Black-oil model	A simplified multiphase-flow formulation (adapted here to CO ₂ -brine two-phase flow) tracking phase pressures and saturations without full compositional detail.
Corner-point grid	The structured-but-geometrically-flexible grid format standard in reservoir simulation, supporting faults and pinch-outs.
Warm-up / startup trials	Initial trials sampled quasi-randomly before the surrogate model has enough data to guide proposals intelligently (36 of 120 here).
Surrogate model	A cheap, fast-to-evaluate statistical approximation (here, Gradient Boosting Regressors for CO ₂ and BHP) fit to accumulated trial data, letting you explore the response surface without running the expensive simulator again.
Feasible / infeasible trial	A trial is feasible only if both physical constraints (BHP and boundary breach) hold; otherwise it is infeasible and penalized in the reward.
Handshake mode	The human-in-the-loop pattern used here: Python proposes and waits (polls) for a human-triggered MATLAB run and its completion signal, rather than calling the simulator directly.
Residual saturation	The trapped minimum saturation of a phase that cannot be further displaced by flow (0.27 brine, 0.20 CO ₂ in this fluid model).

11. Bayesian Optimization vs. Alternative Optimizers

Method	Gradient-free?	Sample-efficient?	Constraint handling	Mixed int/cont?	Notes for this problem
Bayesian Optimization / TPE (chosen)	Yes	Yes — designed for exactly this	Yes, via <code>constraints_func</code> + reward shaping	Yes	Learns a cheap surrogate; spends its limited budget where it is most informative.
Grid Search	Yes	No — exponential cost	No (post-hoc only)	Yes	$5^{15} \approx 3 \times 10^{10}$ simulations at 5 levels per parameter — completely infeasible.
Random Search	Yes	Poor — no learning	No (post-hoc only)	Yes	Used within BO as the warm-up phase here, but alone wastes most of a 120-trial budget.
Gradient-based (SQP / L-BFGS)	No	Very good if gradients exist	Yes (natively)	No	Not usable: MRST is not differentiable end-to-end, and integer <code>T_start/T_end</code> don't fit a smooth NLP.
Genetic Algorithms / CMA-ES	Yes	Moderate — needs hundreds to thousands of evals	Via penalty functions	Yes, with encoding	Would likely need several times the trial budget to converge as tightly.
Simulated Annealing	Yes	Moderate-to-poor in high-D	Via penalty functions	Yes	Single-point local search; struggles with 15-D and does not parallelize knowledge the way a surrogate does.
Response-Surface / DoE	Yes	Good for smooth, low-D responses	Post-hoc	Yes	Classic DoE designs scale poorly past 6–8 dimensions — essentially reinventing a cruder BO.
Reinforcement Learning	Yes	Poor — needs thousands+ interactions	Via reward shaping	Yes, with care	Massively sample-inefficient for a problem where each step is a full 1000-year reservoir simulation.

11.1 Why Bayesian Optimization Was Chosen

- Every evaluation is expensive — a trial means a full 1000-year, 29,128-cell MRST/CO2lab simulation plus a manual MATLAB step. With only 120 trials, the optimizer must extract maximum information from every single one; BO's surrogate model is explicitly built to do that.
- No gradients are available — the MRST simulator is not differentiable end-to-end with respect to well schedules, ruling out SQP/interior-point/L-BFGS outright.
- Mixed, moderately high-dimensional space — 15 parameters (10 integer, 5 continuous with step) is exactly the regime where TPE's tree-structured density model is comfortable, and where grid search is already computationally impossible.
- Constraints are nonlinear, expensive, and only known after the fact — BHP and boundary-breach outcomes can only be measured after running the simulator. Optuna's `constraints_func` plus asymmetric reward shaping is a well-established way to teach a Bayesian sampler where the feasible region likely lies.
- The historical record has practical value — BO's explicit probabilistic model naturally supports the 'rebuild from disk every run' design and the downstream surrogate export (Cell 11). Evolutionary or RL approaches would give up a comparable amount of directly reusable surrogate structure.

Trade-off acknowledged: BO's overhead (fitting a surrogate, computing an acquisition strategy) is irrelevant here because it is dwarfed by simulation cost — a 1000-year reservoir simulation costs vastly more wall-clock time than any TPE bookkeeping. In a world with a cheap, differentiable, or purely low-

dimensional objective, a gradient-based or grid-search method could well be simpler and just as effective — but that is not this problem.

12. Current Project State (from the Uploaded Files)

Metric	Value
Total trials completed	120 (status: OK for all)
Feasible trials	34 / 120 (28.3%)
CO ₂ stored range (all trials)	41.0 – 364.85 Mt
Peak BHP range (all trials)	310.2 – 916.3 bar (worst case far exceeds the 334.8 bar limit)
Best feasible trial	#56 — run folder 04_08_2026__15_27
Best feasible CO ₂	196.8 Mt
Best feasible peak BHP	317.94 bar (within the 334.8 bar limit)
Best feasible boundary breaches	0 cells

12.1 Winning Configuration (Trial #56)

Well	Q (Mt/yr)	T_start (yr)	T_end (yr)	Notes
31/01/01	0.00 (disabled)	0	5	Optimizer chose to disable this well
31/1-3 S	0.10	67	78	Late, short injection window
31/05/07	0.25	60	75	Mid-rate, mid-period
31/07/01	0.35	12	69	Early start, long window
31/4-3	1.50	50	90	Dominant injector — highest rate, widest window

12.2 Manual Follow-up Run

The uploaded `well_plan.csv` shows 31/01/01 re-activated at Q = 0.65 Mt/yr, T = [59, 87] — not the optimizer's 'disabled' choice. The uploaded zip (run folder 04_08_2026__20_21) confirms this was actually simulated and reports the same 196.8 Mt / 317.94 bar / 0-breach result. This appears to be a deliberate manual follow-up test of whether re-enabling the 5th well on top of the BO-selected best configuration for the other 4 still holds. It does — BHP stayed within the limit and containment held.

`bo_signal.json` points to an even more recent run folder `well_csvs/04_08_2026__20_28` (timestamp 2026-08-04 20:31:15, status 'done') — one run after the zip snapshot provided. There is at least one more result on disk beyond what is included here that the notebook will pick up on its next Cell 6 rebuild.

Document generated from direct inspection of the uploaded notebook, CSVs, results JSON, signal file, and sample well_csvs/output archive — not from assumptions about a typical CCS/BO pipeline.